

Document Number: P2029R0
Date: 2020-01-11
Audience: CWG
Reply-to: Tom Honermann <tom@honermann.net>

Proposed resolution for core issues 411, 1656, and 2333; numeric and universal character escapes in character and string literals

This paper proposes substantial changes to [\[lex.phases\]](#), [\[lex.ccon\]](#), and [\[lex.string\]](#) intended to address the following core issues as well as several other more minor issues.

- [Core Issue 411: Use of universal-character-name in character versus string literals](#)
- [Core Issue 1656: Encoding of numerically-escaped characters](#)
- [Core Issue 2333: Escape sequences in UTF-8 character literals](#)

This paper follows a prior [proposed resolution](#) that only attempted to address [CWG 2333](#). That proposed resolution was [discussed on the core mailing list](#) and at the [June 24th, 2019 core issues processing teleconference](#). The resolution proposed in this paper attempts to address the feedback provided during those discussions.

The notes for [CWG 2333](#) in the current active issues list (revision 100) state that discussion at the [August 14th, 2017 core issues processing teleconference](#) resulted in a determination that numeric escape sequences in UTF-8 character literals should be ill-formed. The issue has remained in drafting status since then.

SG16 discussed this issue during its [October 17th, 2018 teleconference](#). The SG16 consensus was for a different resolution than is currently described in the active issues list. The SG16 consensus was based on the following observations:

- The current resolution in the active issues list contradicts existing practice. gcc, Clang, and Visual C++ all allow octal and hexadecimal escape sequences in UTF-8 literals.
- Octal and hexadecimal escape sequences in UTF-8 literals are useful for a number of purposes:
 - Embedding null characters: `u8"\0"`
 - Creating ill-formed code unit sequences for testing purposes.
 - Creating Modified UTF-8, CESU-8, and WTF-8 string literals. This entails two abilities:
 - Embedding U+0000 as an overlong UTF-8 sequence: `u8"\xC0\x80"`
 - Embedding lone surrogate code points as individual UTF-8 code unit sequences. For example, encoding U+D800 as `u8"\xED\xA0\x80"`. (Note that use of `\u` escapes specifying surrogate code points is ill-formed).
 - Compatibility with existing log/debug systems that output literals with non-printable characters represented with escapes so as to facilitate copy/paste of such output into code.

SG16 conducted the following poll:

Continue to allow hex and octal escapes that indicate code unit values, requiring only that they fit into the range of the code unit type?

SF	F	N	A	SA
8	1	0	0	0

In the polled question, "Continue" refers to existing implementation behavior; to maintain the current implementation status quo exhibited by gcc, Clang and Visual C++.

The proposed resolution reflects the SG16 consensus.

[CWG 411](#) is addressed by specifying different behavior for character literals vs string literals for characters that are not representable by a single code unit. For example, when the execution character set is UTF-8, `'\u0153'` is conditionally-supported, has type `int` and an implementation-defined value, but `"\u0153"` is a character array of length 3 containing the UTF-8 encoding of U+0153 (LATIN SMALL LIGATURE OE) and a null character (`\xC5\x93\x00`).

[CWG 1656](#) is addressed by clarifying that numeric escape sequences denote code unit values in the execution character set; that the values are not subject to conversion from the encoding of the source file to the execution character set.

Proposed resolution overview

The proposed wording changes are intended to resolve [CWG 411](#), [CWG 1656](#), and [CWG 2333](#) by:

- Clarifying that hexadecimal and octal escape sequences:
 - are valid in `u8`, `u`, and `U` character literals. (CWG 2333)
 - specify values that need not correspond to valid code unit values for the applicable character encoding. (CWG 2333)
 - specify the execution-time value of character literals or individual code unit values of string literals; that the value expressed is not subject to conversion to the applicable execution character set. (CWG 1656)
 - specify values that may result in string literals that are ill-formed according to their applicable character encoding (CWG 2333).
- Clarifying that characters that cannot be specified in a character literal because they require more than one code unit in the applicable character encoding may be specified in string literals. (CWG 411)

The concept of an "associated character encoding" is introduced for character and string literals so as to enable wording to be independent of the particular kind of literal (ordinary, wide or Unicode).

New `basic-c-char`, `basic-s-char`, and `numeric-escape-sequence` grammar productions are proposed in order to simplify wording. The `c-char`, `s-char`, and `escape-sequence` grammar productions are updated to define them in terms of the new grammar productions.

Additionally, the wording updates are intended to:

- Specify behavior for non-ordinary character and string literals when a character lacks representation in the applicable character encoding. (Wording was previously missing)
- Specify behavior for simple-escape-sequences that denote characters that lack representation in the applicable character encoding. (Wording was previously missing)
- Remove wording that states that `wchar_t` is able to represent all members of the execution wide-character set as this contradicts wide spread existing practice (SG16 has an issue tracking this at <https://github.com/sg16-unicode/sg16/issues/9>).
- Modernize the wording with current style preferences and terminology.

Implementation impact

The author intends the proposed wording to reflect existing practice for the gcc and Clang compilers. If this proposal is adopted, neither of those compilers are expected to require updates.

However, there is implementation impact to the Microsoft Visual C++ compiler. Consider the following (C++20) code:

```
constexpr const char8_t c[] = u8"\xc3\x80"; // UTF-8 encoding of U+00C0 {LATIN CAPITAL LETTER A WITH GRAVE}
#if defined(_MSC_VER)
    // Microsoft Visual C++:
    static_assert(c[0] == 0xC3); // UTF-8 encoding of U+00C3 {LATIN CAPITAL LETTER A WITH TILDE}
    static_assert(c[1] == 0x83);
    static_assert(c[2] == 0xC2); // UTF-8 encoding of U+0080 {<control>}
    static_assert(c[3] == 0x80);
    static_assert(c[4] == 0x00); // null
#else
    // Gcc and Clang:
    static_assert(c[0] == 0xC3); // UTF-8 encoding of U+00C0 {LATIN CAPITAL LETTER A WITH GRAVE}
    static_assert(c[1] == 0x80);
    static_assert(c[2] == 0x00); // null
#endif
```

Gcc and Clang encode the hexadecimal escapes as code units in the target (UTF-8) encoding and perform no conversions (consistent with the behavior intended by this proposal). However, Visual C++ considers each hexadecimal escape to specify a code point in the source encoding and encodes each as UTF-8.

The author does not know if the Visual C++ behavior exhibited for UTF-8 literals is intentional or reflective of a defect. The behavior is inconsistent for UTF-8 and UTF-16 literals. For UTF-8 literals, numeric escape sequences that specify values outside the range of `char8_t` are accepted as code point values and encoded as UTF-8. However, for UTF-16 literals, numeric escape sequences that specify values outside the range of `char16_t` are rejected rather than being considered a code point value and encoded as a UTF-16 surrogate pair.

Acknowledgements

Thank you to Jens Maurer and Steve Downey for providing feedback on initial drafts of this paper!

Proposed resolution

These changes are relative to [N4835](#).

The changes to [\[lex.ccon\]](#) and [\[lex.string\]](#) are rather pervasive. For ease of review, unchanged paragraphs in these sections are retained in the wording below. These paragraphs are introduced with "No changes to ..." and are highlighted with a blue background.

These changes do not reflect recent editorial changes made in <https://github.com/cplusplus/draft/pull/2768>; merge conflict resolution will be required.

☐ Hide inserted text

☐ Hide deleted text

Change in [5.2 \[lex.phases\] paragraph 5](#):

Drafting Note: The change of "escape sequence" to "simple-escape-sequence" addresses [CWG 1656](#).

Each ~~basic source character set member~~ [basic-c-char](#), [basic-s-char](#), and [r-char](#) in a character literal or a string literal, as well as each ~~escape sequence~~ [simple-escape-sequence](#) and [universal-character-name](#) in a character literal or a non-raw string literal, is converted to the ~~corresponding member of the execution character set~~ [literal's associated character encoding \(\[lex.ccon\], \[lex.string\]\)](#); ~~if there is no corresponding member, it is converted to an implementation defined member other than the null (wide) character~~ [If a character lacks representation in the associated character encoding, then the character is converted to an implementation-defined character or the literal is ill-formed as specified in \[lex.ccon\] and \[lex.string\]](#). ⁸

Change in [5.2 \[lex.phases\] paragraph 7](#):

Drafting note: This addition duplicates wording in [\[lex.string\]](#), but seems important to include here.

[A null character is appended to every string-literal.](#) White-space characters separating tokens are no longer significant. Each preprocessing token is converted into a token ([\[lex.token\]](#)). The resulting tokens are syntactically and semantically analyzed and translated as a translation unit. [*Note:* The process of analyzing and translating the tokens may occasionally result in one token being replaced by a sequence of other tokens ([\[temp.names\]](#)). — *end note*] It is implementation-defined whether the sources for module units and header units on which the current translation unit has an interface dependency ([\[module.unit\]](#), [\[module.import\]](#)) are required to be available. [*Note:* Source files, translation units and translated translation units need not necessarily be stored as files, nor need there be any one-to-one correspondence between these entities and any external representation. The description is conceptual only, and does not specify any particular implementation. — *end note*]

Change in [5.13.3 \[lex.ccon\]](#):

[character-literal](#):
[encoding-prefix](#)_{opt} ' [c-char-sequence](#) '

[encoding-prefix](#): one of
u8 u U L

[c-char-sequence](#):
[c-char](#)
[c-char-sequence](#) [c-char](#)

[c-char](#):
~~any member of the basic source character set except the single quote `'`, backslash `\`, or new-line character~~
[basic-c-char](#)
[escape-sequence](#)
[universal-character-name](#)

basic-c-char:

any member of the basic source character set except the single-quote ' , backslash \ , or new-line character

escape-sequence:

simple-escape-sequence

octal-escape-sequence

hexadecimal-escape-sequence

numeric-escape-sequence

simple-escape-sequence: one of

\ ' \ " \ ? \ \

\ a \ b \ f \ n \ r \ t \ v

numeric-escape-sequence:

octal-escape-sequence

hexadecimal-escape-sequence

octal-escape-sequence:

\ octal-digit

\ octal-digit octal-digit

\ octal-digit octal-digit octal-digit

hexadecimal-escape-sequence:

\ x hexadecimal-digit

hexadecimal-escape-sequence hexadecimal-digit

Change in 5.13.3 [lex.ccon].paragraph 1:

A character literal *character literal* is one or more characters a c-char-sequence enclosed in single quotes, as in ~~'x' 'v'~~, optionally preceded by ~~u8, u, U, or L~~ an encoding-prefix, as in u8 'w', u 'x', U 'y', or L 'z', respectively.

Change in 5.13.3 [lex.ccon].paragraph 2:

Drafting Note: Wording for multicharacter literals, characters that lack representation in the execution character set, and the value of a character literal has been moved to new paragraphs.

A character literal that does not begin with ~~u8, u, U, or L~~ an encoding-prefix, such as 'v', is an *ordinary character literal*, has type char unless otherwise specified to have type int as described below, and has the encoding of the execution character set ([lex.charset]) as its associated character encoding. An ordinary character literal that contains a single *c-char* representable in the execution character set has type char, with value equal to the numerical value of the encoding of the *c-char* in the execution character set. An ordinary character literal that contains more than one *c-char* is a *multicharacter literal*. A multicharacter literal, or an ordinary character literal containing a single *c-char* not representable in the execution character set, is conditionally supported, has type int, and has an implementation-defined value.

Change in 5.13.3 [lex.ccon].paragraph 3:

Drafting Note: Wording for multicharacter UTF-8 literals, characters that cannot be represented in a single UTF-8 code unit, and the value of a UTF-8 character literal has been moved to new paragraphs.

A character literal that begins with u8, such as u8 'w', is a ~~character literal of type char8_t, known as a~~ UTF-8 character literal, has type char8_t, and has UTF-8 as its associated character encoding. The value of a UTF-8 character literal is equal to its ISO/IEC 10646 code point value, provided that the code point value can be encoded as a single UTF-8 code unit. [*Note:* That is, provided the code point value is in the range 0x0-0x7F (inclusive). — *end note*] If the value is not representable with a single UTF-8 code unit, the program is ill-formed. A UTF-8 character literal containing multiple *c-chars* is ill-formed.

Change in 5.13.3 [lex.ccon].paragraph 4:

Drafting Note: Wording for multicharacter UTF-16 literals, characters that cannot be represented in a single UTF-16 code unit, and the value of a UTF-16 character literal has been moved to new paragraphs.

A character literal that begins with the letter u, such as u 'x', is a ~~character literal of type char16_t, known as a~~ UTF-16 character literal, has type char16_t, and has UTF-16 as its associated character encoding. The value of a UTF-16 character literal is equal to its ISO/IEC 10646 code point value, provided that the code point value is

representable with a single 16-bit code unit. [*Note:* That is, provided the code point value is in the range 0x0-0xFFFF (inclusive). — *end note*] If the value is not representable with a single 16-bit code unit, the program is ill-formed. A UTF-16 character literal containing multiple *c-chars* is ill-formed.

Change in 5.13.3 [lex.ccon].paragraph 5:

Drafting Note: Wording for multicharacter UTF-32 literals, and the value of a UTF-32 character literal has been moved to new paragraphs.

A character literal that begins with the letter U, such as U'y', is a character literal of type `char32_t`, known as a *UTF-32 character literal*, has type `char32_t`, and has UTF-32 as its associated character encoding. The value of a UTF-32 character literal containing a single *c-char* is equal to its ISO/IEC 10646 code point value. A UTF-32 character literal containing multiple *c-chars* is ill-formed.

Change in 5.13.3 [lex.ccon].paragraph 6:

Drafting Note: Wording for multicharacter wide literals, and the value of a wide character literal has been moved to new paragraphs. The note regarding the ability for `wchar_t` to store all values of the execution wide-character set is intentionally removed as it conflicts with long standing existing practice (<https://github.com/sg16-unicode/sg16/issues/9>).

A character literal that begins with the letter L, such as L'z', is a *wide-character literal*, has type `wchar_t`¹⁸, and has the encoding of the execution wide-character set ([lex.charset]) as its associated character encoding. A wide-character literal has type `wchar_t`⁴⁰. The value of a wide-character literal containing a single *c-char* has value equal to the numerical value of the encoding of the *c-char* in the execution wide-character set, unless the *c-char* has no representation in the execution wide-character set, in which case the value is implementation-defined. [*Note:* The type `wchar_t` is able to represent all members of the execution wide-character set (see [basic.fundamental]). — *end note*] The value of a wide-character literal containing multiple *c-chars* is implementation-defined.

Add a new paragraph after 5.13.3 [lex.ccon].paragraph 6:

A character literal with a *c-char-sequence* containing more than one *c-char*, as in 'abcd', is a *multicharacter literal*. An ordinary character literal that is a multicharacter literal is conditionally supported, has type `int`, and has an implementation-defined value. A wide-character literal that is a multicharacter literal has an implementation-defined value. A UTF-8 character literal, UTF-16 character literal, or UTF-32 character literal that is a multicharacter literal is ill-formed.

Add another new paragraph (X):

The value of a character literal consisting of a single *basic-c-char*, *simple-escape-sequence*, or *universal-character-name* is the code point value of the specified character encoded according to the character literal's associated character encoding. If the character lacks representation in the associated character encoding, then:

- (X.1) for an ordinary character literal, the literal is conditionally supported, has type `int`, and has an implementation-defined value that is not the null character value.
- (X.2) for a wide-character literal, the literal has an implementation-defined value that is not the null wide character value.
- (X.3) for a UTF-8, UTF-16, or UTF-32 character literal, the literal is ill-formed.

If the character requires multiple code units to be encoded in the associated character encoding, then:

- (X.4) for an ordinary character literal, the literal is conditionally supported, has type `int`, and has an implementation-defined value that is not the null character value.
- (X.5) for a wide-character literal, the literal has an implementation-defined value that is not the null wide character value.
- (X.6) for a UTF-8 or UTF-16 character literal, the literal is ill-formed. [*Note:* UTF-32 never requires multiple code units for an encoded character. — *end note*]

Change in 5.13.3 [lex.ccon].paragraph 7:

Drafting Note: Wording for additional conditionally-supported simple escape sequences has been moved to a new paragraph.

The character specified by a *simple-escape-sequence* is specified in table 8. [*Note:* The execution character set and the execution wide-character set do not require representation for the new-line (NL), horizontal tab (HT), vertical tab (VT), or form feed (FF) characters ([lex.charset]); the UTF-8, UTF-16, and UTF-32 encodings

are able to represent all of the *simple-escape-sequence* characters using a single code unit. — end note] Certain non-graphic characters, the single quote `'`, the double quote `"`, the question mark `?`,¹⁹ and the backslash `\`, can be represented according to Table 8. The double quote `"` and the question mark `?` can be represented as themselves or by the escape sequences `\"` and `\?` respectively, but the single quote `'` and the backslash `\` shall be represented by the escape sequences `\'` and `\\` respectively. Escape sequences in which the character following the backslash is not listed in Table 8 are conditionally supported, with implementation-defined semantics. An escape sequence specifies a single character.

Table 8: **E**Simple escape sequences [tab:lex.ccon.esc]

new-line	NL(LF)	<code>\n</code>
horizontal tab	HT	<code>\t</code>
vertical tab	VT	<code>\v</code>
backspace	BS	<code>\b</code>
carriage return	CR	<code>\r</code>
form feed	FF	<code>\f</code>
alert	BEL	<code>\a</code>
backslash	<code>\</code>	<code>\\</code>
question mark ¹⁹	<code>?</code>	<code>\?</code>
single quote	<code>'</code>	<code>\'</code>
double quote	<code>"</code>	<code>\"</code>
octal number	<code>ooo</code>	<code>\ooo</code>
hex number	<code>hhh</code>	<code>\xhhh</code>

Add a new paragraph after 5.13.3 [lex.ccon].paragraph 7:

Additional simple escape sequences in which the character following the backslash is not listed in Table 8 are conditionally-supported, with implementation-defined semantics.

Change in 5.13.3 [lex.ccon].paragraph 8:

Drafting Note: Wording describing the form of octal and hexadecimal escape sequences has been removed as redundant; the form is implicit in the grammar.

The value of a character literal consisting of a single *numeric-escape-sequence* is the numeric value of the octal or hexadecimal number. The escape `\ooo` consists of the backslash followed by one, two, or three octal digits that are taken to specify the value of the desired character. The escape `\xhhh` consists of the backslash followed by ~~one or more~~ hexadecimal digits that are taken to specify the value of the desired character. There is no limit to the number of digits in a hexadecimal sequence. A sequence of octal or hexadecimal digits is terminated by the first character that is not an octal digit or a hexadecimal digit, respectively. The value of a character literal is implementation defined if it falls outside of the implementation defined range defined for `char` (for character literals with no prefix) or `wchar_t` (for character literals prefixed by `L`). [Note: If the value of a character literal prefixed by `u`, `us`, or `U` is outside the range defined for its type, the program is ill-formed. — end note] If the numeric value exceeds the range of the character literal type, then:

(8.1) — for an ordinary character literal, the value is implementation-defined.

(8.2) — for a wide character literal, the value is implementation-defined.

(8.3) — for a UTF-8, UTF-16, or UTF-32 character literal, the literal is ill-formed.

Delete 5.13.3 [lex.ccon].paragraph 9:

Drafting Note: The normative text was combined with wording for basic-c-char and simple-escape-sequence above. The deleted note duplicates normative text in 5.2 [lex.phases].paragraph 1.

A *universal character name* is translated to the encoding, in the appropriate execution character set, of the character named. If there is no such encoding, the *universal character name* is translated to an implementation-defined encoding. [Note: In translation phase 1, a *universal character name* is introduced whenever an actual extended character is encountered in the source text. Therefore, all extended characters are described in terms of *universal-*

~~character-names~~ However, the actual compiler implementation may use its own native character set, so long as the same results are obtained. — ~~end note~~ }

Change in 5.13.5 [lex.string]:

string-literal:

encoding-prefix_{opt} " s-char-sequence_{opt} "
encoding-prefix_{opt} R raw-string

s-char-sequence:

s-char
s-char-sequence s-char

s-char:

~~any member of the basic source character set except the double-quote ", backslash \, or new-line character~~

basic-s-char

escape-sequence

universal-character-name

basic-s-char:

any member of the basic source character set except the double-quote ", backslash \, or new-line character

raw-string:

" d-char-sequence_{opt} (r-char-sequence_{opt}) d-char-sequence_{opt} "

r-char-sequence:

r-char
r-char-sequence r-char

r-char:

any member of the source character set, except a right parenthesis) followed by the initial d-char-sequence (which may be empty) followed by a double quote ".

d-char-sequence:

d-char
d-char-sequence d-char

d-char:

any member of the source character set except:
space, the left parenthesis (, the right parenthesis), the backslash \, and the control characters representing horizontal tab, vertical tab, form feed, and newline.

Change in 5.13.5 [lex.string] paragraph 1:

A string-literal is a sequence of characters (as defined in [lex.econ]) surrounded by double quotes, optionally prefixed by R, u8, u8R, u, uR, U, UR, L, or LR, as in " ", R" ", u8" ", u8R" ", u" ", uR" ", U" ", UR" ", L" ", or LR" ", respectively. ~~is one of the following optionally preceded by an encoding-prefix:~~

(1.1) an optional s-char-sequence enclosed in double quotes as in "", "abc", u8"def", u"ghi", U"jkl", or L"mno".

(1.2) a raw-string preceded by R as in R"(.)", R"(ABC)", u8R"**(DEF)**", uR"*(GHI)*~", UR"zzz(JKL)zzz", or LR"(MNO)".

No changes to 5.13.5 [lex.string] paragraph 2:

A string-literal that has an R in the prefix is a raw string literal. The d-char-sequence serves as a delimiter. The terminating d-char-sequence of a raw-string is the same sequence of characters as the initial d-char-sequence. A d-char-sequence shall consist of at most 16 characters.

No changes to 5.13.5 [lex.string] paragraph 3:

[Note: The characters '(' and ')' are permitted in a [raw-string](#). Thus, `R"delimiter((a|b))delimiter"` is equivalent to `"(a|b)"`. — end note]

No changes to [5.13.5 \[lex.string\].paragraph 4](#):

[Note: A source-file new-line in a raw string literal results in a new-line in the resulting execution string literal. Assuming no whitespace at the beginning of lines in the following example, the assert will succeed:

```
const char* p = R"(a\  
b  
c)";  
assert(std::strcmp(p, "a\\nb\\nc") == 0);
```

— end note]

No changes to [5.13.5 \[lex.string\].paragraph 5](#):

[Example: The raw string

```
R"(a(  
)\  
a"  
)a"
```

is equivalent to `"\n)\na\n"`. The raw string

```
R"(x = "\"y\"")"
```

is equivalent to `"x = \"\\\"y\\\"\""`. — end example]

Add a new paragraph after [5.13.5 \[lex.string\].paragraph 5](#):

[string-literal](#)s have type "array of n const `CHAR_T`" where `CHAR_T` is the [string-literal](#)'s associated code unit type, and n is the number of code units required to represent the contents of the string after [translation phase 7](#) ([\[lex.phases\]](#)) in the [string-literal](#)'s associated character encoding.

Change in [5.13.5 \[lex.string\].paragraph 6](#):

Drafting note: The wording regarding translation phase 6 has been removed as inaccurate; string contents are not fully known until after phase 7. Wording regarding the translation phase dependency and the literal's type has been moved to the new paragraph above. Wording for storage duration and string literal initialization has been moved to a new paragraph.

~~After translation phase 6, a [string-literal](#) that does not begin with an [encoding-prefix](#) is an ordinary string literal, has type `char` as its associated code unit type, and has the encoding of the execution character set ([\[lex.charset\]](#)) as its associated character encoding. An ordinary string literal has type "array of n const `char`" where n is the size of the string as defined below, has static storage duration ([\[basic.stc\]](#)), and is initialized with the given characters.~~

Change in [5.13.5 \[lex.string\].paragraph 7](#):

Drafting note: Wording regarding the literal's type and initialization has been moved to new paragraphs.

A [string-literal](#) that begins with `u8`, such as `u8"asdf"`, is a *UTF-8 string literal*, has type `char8_t` as its associated code unit type, and has UTF-8 as its associated character encoding. ~~A UTF-8 string literal has type "array of n const `char8_t`", where n is the size of the string as defined below; each successive element of the object representation ([\[basic.types\]](#)) has the value of the corresponding code unit of the UTF-8 encoding of the string.~~

No changes to [5.13.5 \[lex.string\].paragraph 8](#):

Ordinary string literals and UTF-8 string literals are also referred to as narrow string literals.

Change in [5.13.5 \[lex.string\].paragraph 9](#):

Drafting note: Wording regarding the literal's type and initialization has been moved to new paragraphs. The note has been deleted as redundant; the use of surrogate pairs is explicit in the UTF-16 encoding.

A [string-literal](#) that begins with `u`, such as `u"asdf"`, is a *UTF-16 string literal*, has type `char16_t` as its associated code unit type, and has UTF-16 as its associated character encoding. ~~A UTF-16 string literal has type "array of n const `char16_t`", where n is the size of the string as defined below; each successive element of the array has the value of the corresponding code unit of the UTF-16 encoding of the string. [Note: A single `c-char` may produce~~

more than one `char16_t` character in the form of surrogate pairs. A surrogate pair is a representation for a single code point as a sequence of two 16-bit code units. — end note]

Change in 5.13.5 [lex.string].paragraph 10:

Drafting note: Wording regarding the literal's type and initialization has been moved to new paragraphs.

A [string-literal](#) that begins with `u`, such as `U"asdf"`, is a *UTF-32 string literal*, has type `char32_t` as its associated code unit type, and has UTF-32 as its associated character encoding. A UTF-32 string literal has type "array of n `const char32_t`", where n is the size of the string as defined below; each successive element of the array has the value of the corresponding code unit of the UTF-32 encoding of the string.

Change in 5.13.5 [lex.string].paragraph 11:

Drafting note: Wording regarding the literal's type and initialization has been moved to new paragraphs.

A [string-literal](#) that begins with `L`, such as `L"asdf"`, is a *wide string literal*, has type `wchar_t` as its associated code unit type, and has the encoding of the execution wide-character set ([lex.charset]) as its associated character encoding. A wide string literal has type "array of n `const wchar_t`", where n is the size of the string as defined below; it is initialized with the given characters.

Change in 5.13.5 [lex.string].paragraph 12:

In [translation phase 6](#) ([lex.phases]), adjacent [string-literals](#) are concatenated. If both [string-literals](#) have the same [encoding-prefix](#), the resulting concatenated string literal has that [encoding-prefix](#). If one [string-literal](#) has no [encoding-prefix](#), it is treated as a [string-literal](#) of the same [encoding-prefix](#) as the other operand. If a UTF-8 string literal token is adjacent to a wide string literal token, the program is ill-formed. Any other concatenations are conditionally-supported with implementation-defined behavior. [Note: This concatenation is an interpretation, not a conversion. Because the interpretation happens in translation phase 6 (after each character from a string literal has been translated into a value from the appropriate character set after the string literal contents have been encoded in the string-literal's associated character encoding), a [string-literal](#)'s initial rawness has no effect on the interpretation or well-formedness of the concatenation. — end note] Table 9 has some examples of valid concatenations.

Table 9: String literal concatenations [tab:lex.string.concat]

Source	Means	Source	Means	Source	Means
<code>u"a" u"b"</code>	<code>u"ab"</code>	<code>U"a" U"b"</code>	<code>U"ab"</code>	<code>L"a" L"b"</code>	<code>L"ab"</code>
<code>u"a" "b"</code>	<code>u"ab"</code>	<code>U"a" "b"</code>	<code>U"ab"</code>	<code>L"a" "b"</code>	<code>L"ab"</code>
<code>"a" u"b"</code>	<code>u"ab"</code>	<code>"a" U"b"</code>	<code>U"ab"</code>	<code>"a" L"b"</code>	<code>L"ab"</code>

Characters in concatenated strings are kept distinct.

[Example:

`"\xA" "B"`

contains the two characters `'\xA'` and `'B'` after concatenation (and not the single hexadecimal character `'\xAB'`). — end example]

Change in 5.13.5 [lex.string].paragraph 13:

After any necessary concatenation, in [translation phase 7](#) ([lex.phases]), ~~`\0`~~ a *null character* is appended to every string literal so that programs that scan a string can find its end.

Delete 5.13.5 [lex.string].paragraph 14:

Drafting note: This wording has been removed as misleading, incomplete, or redundant. String literal contents do not always have the same meaning as in character literals. The wording regarding single and double quotes is redundant with the grammar. The discussion of string length is unnecessary as string length is determined by encoding.

Escape sequences and [universal-character-names](#) in non-raw string literals have the same meaning as in [character literals](#) ([lex.econ]), except that the single quote `'` is representable either by itself or by the escape sequence `\'`, and the double quote `"` shall be preceded by a `\`, and except that a [universal-character-name](#) in a UTF-16 string literal may yield a surrogate pair. In a narrow string literal, a [universal-character-name](#) may map to more than one `char` or `char8_t` element due to [multibyte encoding](#). The size of a `char32_t` or wide string literal is the total number of escape sequences, [universal-character-names](#), and other characters, plus one for the terminating `U'\0'` or `L'\0'`.

The size of a UTF-16 string literal is the total number of escape sequences, [universal-character-names](#), and other characters, plus one for each character requiring a surrogate pair, plus one for the terminating `u'\0'`. [*Note:* The size of a `char16_t` string literal is the number of code units, not the number of characters. — *end note*] Within `char32_t` and `char16_t` string literals, any [universal-character-names](#) shall be within the range `0x0` to `0x10FFFF`. The size of a narrow string literal is the total number of escape sequences and other characters, plus at least one for the multibyte encoding of each [universal-character-name](#), plus one for the terminating `'\0'`.

Change in [5.13.5 \[lex.string\].paragraph 15](#):

Drafting note: Wording for string literal object initialization has been moved to a new paragraph.

Evaluating a [string-literal](#) results in a string literal object with static storage duration ([\[basic.stc\]](#)), ~~initialized from the given characters as specified above~~. Whether all string literals are distinct (that is, are stored in nonoverlapping objects) and whether successive evaluations of a [string-literal](#) yield the same or a different object is unspecified. [*Note:* The effect of attempting to modify a string literal is undefined. — *end note*]

Add a new paragraph (X) after [5.13.5 \[lex.string\].paragraph 15](#):

String literal objects are initialized with a sequence of code unit values corresponding to the sequence of [s-chars](#) (for a non-raw string literal) and [r-chars](#) (for a raw string literal) in the [string-literal](#). The characters denoted by each [basic-s-char](#), [r-char](#), [simple-escape-sequence](#) ([\[lex.ccon\]](#)), and [universal-character-name](#) ([\[lex.charset\]](#)) are encoded to a code unit sequence using the [string-literal](#)'s associated encoding. If a character lacks representation in the associated character encoding, then:

- (X.1) — for an ordinary string literal, an implementation-defined character other than the null character is substituted.
- (X.2) — for a wide string literal, an implementation-defined character other than the null wide character is substituted.
- (X.3) — for a UTF-8, UTF-16, or UTF-32 string literal, the literal is ill-formed.

Each [numeric-escape-sequence](#) ([\[lex.ccon\]](#)) contributes a single code unit value with the numeric value of the octal or hexadecimal number. There is no limit to the number of digits in a hexadecimal sequence. A sequence of octal or hexadecimal digits is terminated by the first character that is not an octal digit or a hexadecimal digit, respectively. If the numeric value exceeds the range of the code unit type, then:

- (X.4) — for an ordinary string literal, an implementation-defined value is substituted.
- (X.5) — for a wide string literal, an implementation-defined value is substituted.
- (X.6) — for a UTF-8, UTF-16, or UTF-32 string literal, the literal is ill-formed.